

LiveDesk

Real-Time Support Chat & Helpdesk Queue - Software Requirements & Development Guide

2. Executive Summary
3. Problem Statement
4. Goals and Objectives
5. Scope
6. Target Users
7. Functional Requirements
8. Non-Functional Requirements
9. Technology Stack
10. High-Level Architecture
11. Complete Project Folder Structure
12. Database Design (ER Overview)
13. Database Schema
14. API Design
15. Authentication & Authorization
16. Application Workflow
17. Business Logic
18. Validation Rules
19. Exception Handling
20. Security
21. Logging
22. Monitoring
23. Scheduler / Background Jobs
24. Notifications
25. Caching Strategy
26. File Storage
27. Third-Party Integrations
28. Deployment
29. CI/CD Pipeline
30. Testing Strategy
31. Coding Standards
32. Git Strategy
33. Milestone-Based Development Plan
34. Risks
35. Future Enhancements
36. Learning Resources
37. Development Checklist
38. Appendix

Project Name: LiveDesk **Tagline:** Real-time customer support chat with intelligent queue routing - built for teams who need live support without enterprise SaaS pricing. **Version:** v1.0 **Document Type:** Software Requirements & Development Guide (SRS + HLD + LLD + Onboarding Guide) **Date:** July 2026

2. Executive Summary

LiveDesk is a real-time customer support platform where customers start a live chat, get automatically routed to an available support agent, and both sides see messages, typing status, and queue position update instantly - no page refresh.

It exists because small and mid-size product teams need live support infrastructure but can't justify enterprise pricing for tools like Zendesk or Intercom, and most in-house alternatives are basic ticket forms with no real-time layer.

Who uses it: end customers seeking help, support agents handling live conversations, and managers/admins monitoring queue health and team performance.

Problem solved: email-based or form-based support has high latency (hours, not seconds) and no visibility into "is anyone actually available right now." LiveDesk closes that gap with live presence, live queueing, and live escalation.

Business value: faster first-response time directly correlates with customer satisfaction and retention; a visible, real-time queue also lets support managers staff appropriately and catch SLA breaches before customers churn.

3. Problem Statement

Existing problems: - Email/ticket-form support has no real-time feedback; customers don't know if anyone has seen their issue. - Agents can't see queue load at a glance, leading to uneven workload and missed SLAs. - Existing open-source helpdesk tools are either too heavyweight (full CRM suites) or too primitive (static contact forms).

Current limitations: most student/portfolio-level "support" projects are simple contact forms with no live component, and most production tools are closed-source SaaS with recurring cost.

Motivation: build a lean, real-time-first support tool that solves the actual bottleneck - visibility and speed - rather than adding CRM bloat.

4. Goals and Objectives

Primary goals - Real-time chat between customer and agent with sub-second message delivery - Automatic ticket routing to an available agent based on load - Live queue visibility for both customers ("you are #3 in queue") and agents

Secondary goals - SLA-based auto-escalation when a ticket goes unanswered - Agent presence and "typing..." indicators - Basic analytics: average response time, tickets resolved per agent

Success criteria / KPIs - Message delivery latency under 500ms end-to-end - Ticket auto-assignment within 2 seconds of creation - Zero message loss on agent reconnect after a dropped WebSocket connection

5. Scope

In Scope (v1.0) - Customer chat widget (basic web UI) and agent dashboard - Real-time messaging via WebSocket - Queue management and round-robin/load-based routing - Ticket lifecycle: open, assigned, in-progress, resolved, closed - SLA timers and escalation notifications - Basic authentication for agents/admins (JWT); customers join via a generated session, no signup required - Chat history persistence

Out of Scope (deferred to v2+) - Multi-channel support (email, SMS, social) - chat only in v1 - AI-assisted auto-replies or chatbots - Full CRM features (contact management, deal tracking) - Native mobile apps (web-responsive only)

6. Target Users

Role	Responsibilities	Permissions	Key Features
Customer	Starts a chat, describes issue	No login required (session-based)	Chat widget, queue position, chat history for their session
Support Agent	Handles assigned chats, resolves tickets	Login required; sees only assigned/available tickets	Live queue view, chat interface, canned responses, mark resolved
Admin / Manager	Oversees team, monitors SLAs	Full access; manage agents, view analytics	Dashboard, agent management, SLA config, reporting

7. Functional Requirements

7.1 Feature: Start a Chat (Customer)

- **Description:** Customer opens the chat widget and sends an initial message, creating a ticket.
- **Workflow:** widget generates anonymous session -> customer sends message -> backend creates Ticket (status OPEN) and ChatSession -> routing engine assigns an agent or places in queue.
- **Validation:** message body 1-2000 characters, not empty/whitespace-only.
- **Business Rules:** if no agent is available, ticket enters QUEUED status with a position number.
- **Success Response:** 201 Created with ticketId, sessionToken, queue position (if queued).
- **Failure Response:** 400 on empty message; 503 if no agents configured in system at all.
- **Edge Cases:** customer refreshes page mid-chat (session token must allow reconnect), customer sends multiple rapid messages before assignment (must not create duplicate tickets).

7.2 Feature: Ticket Routing / Auto-Assignment

- **Description:** When a ticket is created or an agent becomes free, the routing engine assigns the oldest queued ticket to the least-loaded available agent.
- **Inputs:** ticket queue, agent availability status, current active-chat count per agent.
- **Outputs:** ticket assignedAgentId updated, both parties notified via WebSocket.
- **Business Rules:** an agent is “available” if online and active-chat count is below their configured max concurrency (default 3).
- **Error Cases:** all agents at max capacity -> ticket stays queued,

position broadcast to customer.

7.3 Feature: Real-Time Messaging

- **Description:** Messages sent by either party appear instantly for the other, with delivery/read status.
- **Workflow:** message sent via WebSocket /app/chat/{ticketId} -> persisted to DB -> broadcast to /topic/chat/{ticketId} -> both clients render it.
- **Validation:** sender must be a participant of that ticket (customer session owner or assigned agent).
- **Edge Cases:** agent reconnects after a dropped connection - must fetch missed messages since last-seen timestamp, not just resume the live stream.

7.4 Feature: Typing Indicator & Presence

- **Description:** Shows “agent is typing” and whether the other party is currently online.
- **Workflow:** ephemeral WebSocket event (not persisted) broadcast on keystroke debounce (every 2s while typing).

7.5 Feature: SLA Escalation

- **Description:** If a queued ticket isn’t assigned within N minutes, or an assigned ticket has no agent reply within N minutes, it’s flagged and escalated (notify a manager).
- **Workflow:** scheduled job runs every minute, checks Ticket.lastActivityAt and Ticket.status against configured SLA thresholds.
- **Business Rules:** escalation raises Ticket.priority and sends a notification (email + in-app) to managers.

7.6 Feature: Resolve / Close Ticket

- **Description:** Agent marks a ticket resolved; customer can reopen within a grace window (e.g., 24h) which routes back to the same agent if still online.

8. Non-Functional Requirements

Category	Requirement
Performance	Message delivery under 500ms; API p95 latency under 200ms
Scalability	Horizontally scalable app servers via Redis pub/sub for WebSocket fan-out
Availability	Target 99.5% uptime for chat service
Security	JWT-secured agent/admin endpoints, session-token-scoped customer access, rate limiting on chat creation
Reliability	No message loss on reconnect; persisted before broadcast, not after
Maintainability	Layered architecture, documented API, test coverage on routing/escalation logic

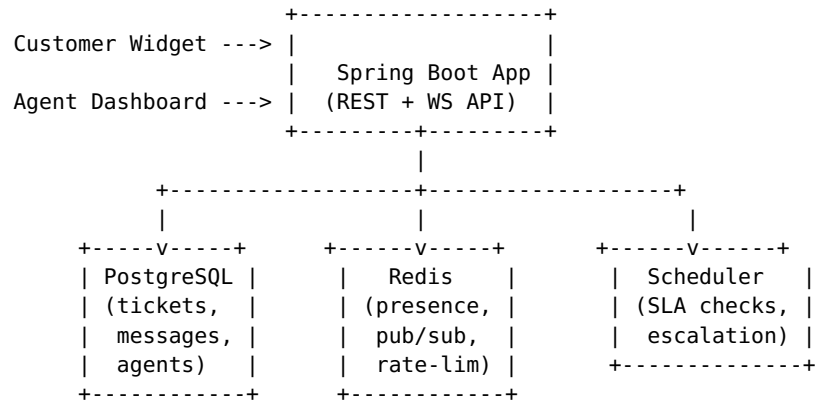
Accessibility	Widget usable via keyboard navigation, basic screen-reader labels
Usability	Queue position and estimated wait visible to customer at all times
Logging	Structured JSON logs, correlation ID per ticket/session
Monitoring	Health checks, queue depth metric, agent utilization metric
Caching	Redis for active session/presence state, agent availability cache
Rate Limiting	Per-IP limit on ticket creation to prevent spam/abuse
Backup/Recovery	Daily PostgreSQL backups, point-in-time recovery enabled

9. Technology Stack

Layer	Technology	Why
Backend	Spring Boot (Java)	Mature ecosystem for REST + WebSocket in one app
Real-time	WebSocket (STOMP/SockJS)	Instant bidirectional messaging
Database	PostgreSQL	Relational integrity for tickets/agents/messages
ORM	Spring Data JPA + Hibernate	Standard persistence layer for Spring Boot
Cache / Pub-Sub	Redis	Presence state, WebSocket fan-out across instances, rate-limit counters
Auth	JWT (agents/admins), signed session tokens (customers)	Different trust levels need different auth strengths
Scheduling	Spring @Scheduled	SLA escalation checks
Notifications	JavaMail/SMTP (email), WebSocket (in-app)	Escalation alerts to managers
API Docs	springdoc-openapi (Swagger UI)	Auto-generated, always-current API docs
Containerization	Docker + Docker Compose	Local parity with deployment
CI/CD	GitHub Actions	Automated test + build on PR
Testing	JUnit 5, Mockito, Testcontainers	Unit + integration tests against real Postgres/Redis in CI
Monitoring	Spring Boot Actuator + Micrometer + Prometheus + Grafana	Health, metrics, dashboards
Logging	SLF4J + Logback (JSON encoder)	Structured, parseable logs

10. High-Level Architecture

Pattern: Modular monolith, layered architecture (Controller -> Service -> Repository), with a distinct routing and scheduler module to keep business logic isolated from transport concerns.



Not microservices - a single deployable app is the right call at this scale; the architecture is kept clean internally so it *could* split into services later (chat-service, routing-service) without a rewrite.

11. Complete Project Folder Structure

```
src/main/java/com/livedesk/
  controller/      REST controllers (TicketController,
AgentController, AdminController)
  websocket/       STOMP config, WebSocket event listeners
(connect/disconnect)
  service/         Business logic (TicketService, RoutingService,
EscalationService)
  repository/      Spring Data JPA repositories
  entity/          JPA entities (Ticket, Message, Agent,
ChatSession)
  dto/             Request/response DTOs
  mapper/          Entity <-> DTO mapping (MapStruct)
  config/          Security, WebSocket, Redis, OpenAPI config
  security/        JWT filter, session-token validation
  scheduler/       SLA escalation job
  events/          Application events (TicketAssignedEvent,
MessageSentEvent)
  exception/       Custom exceptions + global handler
  validation/      Custom validators
  util/            Helpers (ID generation, time utils)
  constant/        Enums, constants (TicketStatus, Roles)
src/main/resources/
  application.yml
  db/migration/    Flyway migration scripts
src/test/java/...  Mirrors main structure
```

12. Database Design (ER Overview)

Entities: Agent, Ticket, Message, ChatSession, SLAConfig

Relationships: - Agent (1) — (N) Ticket : one agent handles many tickets - Ticket (1) — (N) Message : one ticket has many messages - Ticket (1) — (1) ChatSession : each ticket tied to one customer session

Agent ||--o{ Ticket : "handles"
Ticket ||--o{ Message : "contains"
Ticket ||--|| ChatSession : "belongs to"

Constraints: Ticket.assignedAgentId nullable (queued tickets have none); Message.senderType enum (CUSTOMER/AGENT); composite index on (Ticket.status, Ticket.createdAt) for queue ordering.

13. Database Schema

Table: agents | Field | Type | Nullable | Unique | Default | Notes | |
--|--|--|--| id | UUID | No | Yes (PK) | gen_random_uuid() | | name | VARCHAR(100) | No | No | - | | email | VARCHAR(150) | No | Yes | - | login identifier | | password_hash | VARCHAR(255) | No | No | - | BCrypt | | max_concurrency | INT | No | No | 3 | | is_online | BOOLEAN | No | No | false | | role | VARCHAR(20) | No | No | 'AGENT' | AGENT / ADMIN |

Table: tickets | Field | Type | Nullable | Notes | |
--|--|--|--| id | UUID | No | PK | | status | VARCHAR(20) | No | OPEN/QUEUED/ASSIGNED/RESOLVED/CLOSED | | priority | VARCHAR(10) | No | LOW/NORMAL/HIGH/ESCALATED | | assigned_agent_id | UUID | Yes | FK -> agents.id | | created_at | TIMESTAMP | No | index | | last_activity_at | TIMESTAMP | No | used by SLA job |

Table: messages | Field | Type | Nullable | Notes | |
--|--|--|--| id | UUID | No | PK | | ticket_id | UUID | No | FK -> tickets.id, indexed | | sender_type | VARCHAR(10) | No | CUSTOMER/AGENT | | body | TEXT | No | | sent_at | TIMESTAMP | No | index |

Table: chat_sessions | Field | Type | Nullable | Notes | |
--|--|--|--| id | UUID | No | PK | | ticket_id | UUID | No | FK, unique | | session_token | VARCHAR(255) | No | signed, unique | | created_at | TIMESTAMP | No | |

14. API Design

POST /api/v1/tickets - create a ticket (customer) - Auth: none (public, rate-limited by IP) - Body: { "message": "string" } - Response 201: { "ticketId", "sessionToken", "queuePosition" } - Errors: 400 invalid body, 429 rate-limited

GET /api/v1/tickets/{id} - fetch ticket + history - Auth: session token (customer) or JWT (agent, must be assigned) - Response 200: ticket details + message list - Errors: 403 not authorized for this ticket, 404 not found

POST /api/v1/agents/login - agent authentication - Body: { "email", "password" } - Response 200: { "accessToken", "refreshToken" } - Errors: 401 invalid credentials

PATCH /api/v1/tickets/{id}/resolve - mark resolved (agent only) - Auth: JWT, must be the assigned agent - Response 200: updated ticket - Errors: 403 not the assigned agent, 409 already resolved

WebSocket destinations: - /app/chat/{ticketId} (client sends message) -> /topic/chat/{ticketId} (broadcast) - /app/typing/{ticketId} -> /topic/typing/{ticketId} (ephemeral, not persisted) - /topic/queue -> broadcasts live queue depth to agent dashboards

15. Authentication & Authorization

- **Agents/Admins:** JWT access token (15 min expiry) + refresh token (7 days, rotated on use), password hashed with BCrypt.
 - **Customers:** no account; a signed, opaque session token is issued on ticket creation and scopes access to only that ticket's data - short-lived but reconnect-friendly.
 - **RBAC:** two roles, AGENT and ADMIN. Admin-only endpoints (agent management, SLA config, analytics) enforced via `@PreAuthorize("hasRole('ADMIN')")`.
 - **WebSocket auth:** JWT/session token passed during STOMP CONNECT handshake; subscription to a ticket's topic is rejected server-side unless the connecting principal owns/is assigned to that ticket.
-

16. Application Workflow

```
Customer opens widget -> sends first message
|
Ticket created (status: OPEN)
|
Routing engine checks agent availability
|
+---- Agent available? ----+
| YES                        | NO
v                            v
Assign agent                Queue ticket, broadcast position
|                            |
Live chat begins <----- Agent frees up -> auto-assign
|
Agent resolves ticket -> status RESOLVED
|
Customer reopens within 24h? -- YES --> reassign to same agent if
online
| NO
Ticket auto-closes after grace window
```

17. Business Logic

- **Routing algorithm:** least-active-chats-first among online agents; ties broken by longest-idle agent (fairness).
 - **Priority calculation:** starts NORMAL; escalation job bumps to ESCALATED if unassigned > 5 min or unanswered > 3 min after assignment.
 - **Reopen logic:** resolved tickets reopened within 24h route back to the original agent if still online, otherwise re-enter the normal queue.
 - **Escalation notification:** on escalation, notify all online ADMIN-role users via WebSocket + email fallback if none are online.
-

18. Validation Rules

- **Frontend:** message length limits, disallow empty submission, basic profanity/spam pattern warning (non-blocking).
 - **Backend:** re-validate message length/content server-side (never trust client), ticket ownership check on every request.
 - **Database:** NOT NULL constraints on required fields, foreign key constraints enforced, enum-backed status columns.
 - **Security validations:** session token signature verification, JWT signature + expiry check on every protected request.
-

19. Exception Handling

- Global @ControllerAdvice maps exceptions to consistent JSON error bodies ({ "error", "message", "timestamp", "path" }).
 - Custom exceptions: TicketNotFoundException (404), UnauthorizedTicketAccessException (403), AgentAtCapacityException (used internally by routing, not exposed as HTTP error), InvalidSessionTokenException (401).
 - All exceptions logged with correlation ID; 5xx-level exceptions logged at ERROR with stack trace, 4xx at WARN without stack trace (expected client errors shouldn't spam error logs).
-

20. Security

- OWASP Top 10 awareness: parameterized queries via JPA (no raw SQL string concatenation) prevents SQL injection; output encoding on any rendered chat content prevents stored XSS; CSRF not applicable to stateless JWT APIs but WebSocket origin checks enforced.
 - HTTPS enforced in production (redirect HTTP -> HTTPS at load balancer).
 - Secrets (JWT signing key, DB credentials, SMTP credentials) via environment variables, never committed; .env.example provided.
 - Passwords hashed with BCrypt (agent accounts only).
 - CORS restricted to known frontend origins.
 - Rate limiting on public ticket-creation endpoint (per-IP, Redis-backed token bucket) to prevent spam-ticket abuse.
-

21. Logging

- Log levels: DEBUG (local dev only), INFO (normal flow - ticket created, assigned, resolved), WARN (recoverable issues - agent at capacity, reconnect events), ERROR (unhandled exceptions).
 - JSON structured log format with ticketId/correlationId in MDC for traceability across a single conversation's lifecycle.
 - Audit log (separate from app logs): agent login/logout, ticket reassignment, escalation events - retained longer for accountability review.
-

22. Monitoring

- Spring Boot Actuator health endpoint (/actuator/health) checked by container orchestrator.
- Custom Micrometer metrics: queue.depth, agent.utilization,

message.delivery.latency, escalation.count.

- Prometheus scrapes metrics; Grafana dashboard visualizes queue depth and SLA breach rate over time.
 - Alert rule (documented, not necessarily wired to a pager in v1): trigger if queue depth exceeds threshold for 5+ minutes.
-

23. Scheduler / Background Jobs

- **SLA Escalation Job:** runs every 60 seconds, scans tickets in QUEUED/ASSIGNED status against SLA thresholds, escalates and notifies as needed.
 - **Auto-close Job:** runs hourly, closes RESOLVED tickets past their 24h reopen window.
 - **Retry logic:** email notification sends retried up to 3 times with exponential backoff; failures logged, not silently dropped.
-

24. Notifications

- **Email:** SLA escalation alerts to managers, ticket-resolved confirmation to customer (if email captured).
 - **WebSocket (in-app):** live queue position updates, new-message alerts, agent assignment notification.
 - **Future (v2):** SMS/push notification channel for mobile.
-

25. Caching Strategy

- Redis stores: agent online/offline presence (agent:presence:{id}, TTL-refreshed via heartbeat), live queue depth counter, rate-limit token buckets.
 - Not used for message content caching in v1 - messages are read from PostgreSQL directly since chat history reads are infrequent relative to writes.
 - Cache invalidation: presence keys expire naturally (TTL); no manual eviction needed for this use case.
-

26. File Storage

- v1 does not support file/image attachments in chat (explicitly out of scope) to keep the real-time core focused; documented as a v2 enhancement using S3-compatible object storage with pre-signed upload URLs.
-

27. Third-Party Integrations

Integration	Purpose
SMTP provider (e.g., SendGrid, Mailgun)	Escalation and resolution emails
None required for auth	Agent auth is self-hosted (JWT), no external identity provider in v1

28. Deployment

- **Docker Compose (local/staging):** app + PostgreSQL + Redis + Prometheus + Grafana as services.
 - **Environment variables:** DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET, REDIS_HOST, SMTP_* - injected at container runtime, never baked into the image.
 - **Production build:** multi-stage Dockerfile (Maven build stage -> slim JRE runtime stage) to keep image size small.
 - **Deployment steps:** build image -> push to registry -> deploy on Railway/Render (single instance for portfolio scale) -> run Flyway migrations on startup.
-

29. CI/CD Pipeline

- **Git workflow:** feature branches off main, PR required, at least one self-review checklist item completed before merge.
 - **GitHub Actions pipeline:** on PR - run unit + integration tests (Testcontainers spins up real Postgres/Redis in CI) -> build -> (on merge to main) build and push Docker image.
 - **Rollback:** previous image tag kept available; rollback = redeploy prior tag.
-

30. Testing Strategy

- **Unit tests:** routing algorithm (given agent loads, assert correct assignment), SLA escalation logic (given timestamps, assert correct priority bump).
 - **Integration tests:** full ticket-creation-to-assignment flow with real Postgres/Redis via Testcontainers.
 - **API tests:** REST endpoints via MockMvc/RestAssured, including auth-failure cases.
 - **WebSocket tests:** STOMP test client asserting message broadcast reaches subscribed sessions and rejects unauthorized subscriptions.
 - **Load testing (basic):** simulate N concurrent customers + M agents with a tool like Gatling to validate the 500ms delivery target holds under load.
-

31. Coding Standards

- Java naming conventions (PascalCase classes, camelCase methods/fields), package-by-layer structure as shown in Section 11.
 - SOLID principles applied at service-layer boundaries (e.g., RoutingService depends on an AgentAvailabilityProvider interface, not a concrete Redis client, so the routing algorithm is unit-testable in isolation).
 - Design patterns used: Strategy (routing algorithm swap point), Observer (application events for ticket-assigned/message-sent), Template Method (base exception handling).
 - Consistent formatting via a shared .editorconfig / Spotless/Checkstyle config enforced in CI.
-

32. Git Strategy

- **Branching:** main (always deployable), feature/* branches per unit of work, short-lived.
 - **Commit messages:** Conventional Commits style (feat:, fix:, refactor:, test:, docs:).
 - **PRs:** required before merge, description must state what changed and why.
 - **Tags:** v1.0.0 on first deployable release, semantic versioning thereafter.
-

33. Milestone-Based Development Plan

Phase	Focus	Estimated Time
Phase 1	Project setup, entity model, basic ticket CRUD	1 week
Phase 2	Agent auth (JWT), session-token flow for customers	1 week
Phase 3	WebSocket messaging (core chat)	1.5 weeks
Phase 4	Routing engine + queue logic	1 week
Phase 5	SLA escalation scheduler + notifications	1 week
Phase 6	Presence, typing indicators, reconnect handling	1 week
Phase 7	Testing (unit/integration/WebSocket), hardening	1 week
Phase 8	Observability (Actuator/Prometheus/Grafana), deployment, docs	1 week

Total: roughly 8 weeks at a steady, part-time pace.

34. Risks

Risk	Type	Mitigation
WebSocket connections don't scale past one instance	Technical	Redis pub/sub fan-out designed in from Phase 3, not bolted on later
Routing race condition assigns two tickets to a full agent	Technical	Use DB-level locking/atomic increment on agent's active-chat count
Spam ticket creation floods queue	Security	Per-IP rate limiting from day one on the public endpoint
SLA job doesn't run if a single instance crashes	Reliability	Document as known v1 limitation; v2 = distributed scheduler lock (ShedLock)

Underestimating
real-time debugging
time

Business/Schedule

Buffer week included
in Phase 3 estimate

35. Future Enhancements

- Multi-channel support (email-to-chat bridge, SMS)
 - File/image attachments via S3
 - AI-assisted canned-response suggestions for agents
 - Customer satisfaction (CSAT) survey after resolution
 - Multi-team routing (route by department/skill tag)
-

36. Learning Resources

- Spring Boot WebSocket official guide (Spring.io)
 - “Designing Data-Intensive Applications” (Kleppmann) - for queueing/consistency intuition
 - Redis pub/sub official documentation
 - Baeldung Spring Security JWT tutorials
-

37. Development Checklist

- ☐ Project skeleton generated, runs locally
 - ☐ Entities + Flyway migrations for all tables in Section 13
 - ☐ Ticket creation endpoint + validation
 - ☐ Agent login (JWT) working end-to-end
 - ☐ WebSocket chat working between two manually-opened browser tabs
 - ☐ Routing engine assigns correctly under load-tested scenarios
 - ☐ SLA escalation job verified with time-shifted test data
 - ☐ Presence + typing indicators working
 - ☐ Reconnect flow tested (kill connection mid-chat, verify no message loss)
 - ☐ Unit + integration test suite passing in CI
 - ☐ Dockerized, runs via docker compose up
 - ☐ Deployed with a live URL
 - ☐ README complete with architecture diagram and setup steps
-

38. Appendix

Glossary - *SLA*: Service Level Agreement - the time threshold before a ticket is considered breached. - *Presence*: whether a user (agent) currently has an active connection. - *Queue position*: a customer's rank among tickets awaiting assignment.

Useful commands

<code>./mvnw spring-boot:run</code>	<code># run locally</code>
<code>docker compose up -d</code>	<code># start Postgres + Redis</code>
<code>./mvnw test</code>	<code># run test suite</code>
<code>./mvnw flyway:migrate</code>	<code># apply DB migrations manually</code>

Useful SQL queries

```
-- current queue depth
SELECT COUNT(*) FROM tickets WHERE status = 'QUEUED';
```

```
-- average time-to-first-assignment (last 24h)
SELECT AVG(EXTRACT(EPOCH FROM (assigned_at - created_at)))
FROM tickets WHERE created_at > NOW() - INTERVAL '24 hours';
```

Common troubleshooting - WebSocket connects but no messages arrive -> check Redis pub/sub channel subscription on the receiving instance. - Agent stuck “at capacity” incorrectly -> check for a leaked active-chat counter (ticket resolved but counter not decremented - look for a missing @Transactional boundary). - Tests fail only in CI -> almost always a Testcontainers startup timing issue; check container health-check wait strategy.